Ders 7 Genişletilmiş Notu: Veri Akışları: Bloom Filtresi, DGIM ve Dönem Projesi

Ders 7 Genişletilmiş Notu: Veri Akışları: Bloom Filtresi, DGIM ve Dönem Projesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Nerede Kaldık: Bölüm 4'e Giriş
- 2. Akış Verisi Nedir?
- 3. İlişkisel Veritabanı ile Akış Veri Yönetim Sistemi Arasındaki Fark
- 4. Akış Verisi Arşivlenir, Sorgulanmaz
- 5. Anlık Sorgular ve Sürekli Sorgular
- 6. Akış Veri Yönetim Sisteminin Yapısı
- 7. Uygulamalar: Sorgu Akışları, Tıklama Akışları, IP Paketleri
- 8. Kayan Pencere
- 9. Pencere Belleğe Sığmazsa: Ortalamayı Güncellemek
- 10. Bloom Filtresi: Problem ve Fikir
- 11. Bloom Filtresine Eleman Ekleme: Oyuncak Örnek
- 12. Bloom Filtresinde Sorgulama ve Yanlış Pozitif
- 13. Yanlış Pozitif Olasılığı: Dart Modeli
- 14. Son k Bitteki 1'leri Saymak
- 15. İlk Fikir: Üstel Büyüyen Bloklar
- 16. DGIM Algoritması
- 17. DGIM'de Sorgu ve Hata Sınırı
- 18. Dönem Projesinin Teknik Konusu
- Örnek Sorular
- Dersin Toparlaması

1. Nerede Kaldık: Bölüm 4'e Giriş

Ders online yapıldı. Hoca her zamanki gibi iki ayrı hesapla bağlandığını söyledi: biriyle sunumu paylaşıyor, diğeriyle ses ve görüntüsünü gönderiyordu. Ders büyük ölçüde İngilizce anlatıldı, bazı kısımlar Türkçe tekrar edildi.

Hoca, ders kitabının (Mining of Massive Datasets) 1, 2, 3 ve 7. bölümlerinin işlendiğini hatırlattı. Bu derste 4. bölüme, yani veri akışlarında madenciliğe (mining data streams) geçildi. Bölüm anlatıldıktan sonra dersin son kısmında dönem projesi konuşuldu.

2. Akış Verisi Nedir?

Akış verisi (stream data), sisteme sürekli ve durmaksızın gelen veridir. Hocanın ilk örneği bir sensördü: Ekranda bir dizi tam sayı vardı ve bu sayıların bir sensör tarafından kesintisiz gönderildiğini düşünmemiz istendi. Bu sensör sismik bir sensör, bir sıcaklık sensörü ya da nem sensörü olabilir. Veri hiç durmadan geliyorsa, bu sürekli gelen veriyi analiz edebilen bir yazılıma ihtiyaç vardır.

3. İlişkisel Veritabanı ile Akış Veri Yönetim Sistemi Arasındaki Fark

Hoca, diğer derslerde görülen ilişkisel veritabanı yönetim sistemleriyle (relational database management systems) karşılaştırma yaptı. İlişkisel veritabanında veri sabit diskte tutulur. Veriyi eklemek için SQL `INSERT` komutları kullanılır. Verinin veritabanına ne zaman ve nasıl konulacağını veritabanı yöneticisi ya da kodu yazan geliştirici kontrol eder. Yani girişin kontrolü geliştiricinin elindedir.

Akış veri yönetim sisteminde (stream data management system) durum farklıdır. Sisteme gelen veriyi siz ya da geliştirici kontrol etmezsiniz; veri dışarıdan kontrol edilir.

- **Google arama sorguları:** Her gün milyonlarca insan arama motoruna sorgu girer. İnsanlar bunu özerk (autonomous) biçimde yapar; üzerlerinde merkezi bir kontrol yoktur. Sorgular günün herhangi bir saatinde, rastgele gelir. Bu yüzden sorguların giriş hızı (input rate) üzerinde bir kontrol yoktur.
- **Sensör verisi:** Sistem, sensörlerden gelen verinin hızını kontrol etmez.

Özetle: İlişkisel veritabanında veritabanına giriş üzerinde kontrolünüz vardır. Akış veri yönetim sisteminde ne giriş üzerinde ne de giriş hızı üzerinde kontrolünüz vardır.

Ayrıca akış verisi birden fazla giriş noktasından (input port) gelebilir. Hocanın gösterdiği şekilde, bir bilgisayarda çalışan akış veri yönetim sistemine birden fazla port üzerinden, her birinden sürekli olarak veri geliyordu. Akış verisi birden fazla porttan ve yüksek hızla gelebilir.

4. Akış Verisi Arşivlenir, Sorgulanmaz

Hoca, akış verisinde sistemin bütün akışı erişilebilir biçimde saklayamayacağını söyledi ve bunun ne demek olduğunu açıkladı.

Bir veri kümesini ilişkisel veritabanına koyduğunuzda onu operasyonel veritabanı (operational database) olarak kullanmayı umarsınız: güncelleme, silme, ekleme ve sorgulama yaparsınız. Akış verisi de bir veritabanında saklanabilir, ama bu veritabanları genellikle önceki derslerde öğrenilen HDFS gibi dağıtık sistemlerdir ve operasyonel veritabanı değildir. Akış verisi yalnız bir arşivi olsun diye saklanır. Arşivlenmesi gereken akış verisinin miktarı çok büyük olduğu için böyle büyük bir veritabanını sorgu amacıyla kullanmak çok uzun sürer.

Bu durumda şu soru gelir: Arşivdeki akış verisi üzerinde sorgu yapmıyorsak, bu veri hakkında bir şeyi nasıl anlayacağız, nasıl bilgi çıkaracağız? Cevap şudur: Sunucunun ya da iş istasyonunun sınırlı fiziksel belleği üzerinde çalışan akış yönetim sistemleri kullanılır. Fiziksel arşive gidip bilgi çıkarmak yerine gerekli hesaplama bellekte yapılır.

5. Anlık Sorgular ve Sürekli Sorgular

Bellekte sorgu yapmanın iki yolu vardır: anlık sorgular (ad hoc queries) ve sürekli sorgular (standing queries).

5.1 Anlık Sorgular

Anlık sorgu, akış hakkında bir kez sorulan sıradan sorgudur. Örnek: "Akış S'te şu ana kadar görülen en büyük değer nedir?"

Sistemde anlık sorgular için bir arayüz bulunur. Sistem fiziksel bellekte en büyük değeri tutar ve her yeni veri geldiğinde bu değeri günceller. Biri "Şu ana kadar görülen en büyük değer ne?" diye sorduğunda bellekteki bu konumdaki sayı cevap olarak verilir. Aynısı en küçük değer için de yapılabilir. Soru ortalama ya da standart sapma da olabilir. Bütün bu anlık sorgular için sistemde değişkenler tutulur, her yeni veriyle güncellenir ve soru gelince bu değişkenlerin değeri döndürülür.

5.2 Sürekli Sorgular

Sürekli sorgu, akış hakkında her zaman sorulan sorgudur. Örnek: "Akış S'te görülen her yeni en büyük değeri bildir." Bu durumda soru fiziksel belleğe konur. Her yeni veri geldiğinde akış

veri yönetim sistemi bu sorguya başvurur: Bu değer şu ana kadarki yeni en büyük değer mi? Öyleyse sürekli sorgu bir çıktı üretir.

5.3 İki Sorgu Türünün Karşılaştırması: En Büyük Değer Örneği

Hoca bir akış üzerinden iki sorgu türünü yan yana anlattı. Akışın başında 1 vardır, sonra sırasıyla 5, 2 ve 7 gelir. Yeni gelen veri hep sona eklenir.

Gelen değer	Anlık sorgu: bellekteki en büyük	Sürekli sorgu: çıktı
1	1	Var (1)
5	5	Var (5)
2	5 (değişmez, $2 < 5$)	Yok
7	7	Var (7)

Anlık sorguda çıktı üretilmez; yalnız sorulabilecek sorular için bir bellek konumu güncellenir. Çıktı, soru sorulduğu anda üretilir. Sürekli sorguda ise veri geldikçe sorgu hep veriye uygulanır ve sorgunun bir sonucu varsa çıktı üretilip son kullanıcıya bildirilir.

6. Akış Veri Yönetim Sisteminin Yapısı

Hocanın gösterdiği örnek akış veri yönetim sisteminde iki sorgu arayüzü vardı:

- **Anlık sorgu arayüzü:** İstek-yanıt (request-response) sistemidir. İlişkisel veritabanıyla etkileşimde olduğu gibi sorgu cümlesini gönderirsiniz, sistem sonucu döndürür.
- **Sürekli sorgu arayüzü:** Bildirim tabanlı yayınla-abone ol (publish-subscribe) sistemidir. Sürekli sorgunuzu sisteme koyar ve o sorguya abone olursunuz. Sorgunun bir çıktısı olduğunda çıktı o konuya (topic) yayınlanır ve size bildirim olarak gelir.

Sisteme gelen akış verisi bir arşiv deposuna (archival storage) yazılır. Şekilde tek bir veritabanı çizilmiş olsa da bu arşivler genellikle dağıtık veritabanlarıdır; akış verisini birden fazla sunucu saklar. Bunlar operasyonel değildir, genellikle üzerlerinde sorgu yapılmaz.

Sistemde ayrıca sınırlı bir çalışma belleği (limited working storage) vardır. Fiziksel bellekte iki tür bilgi tutulur: sürekli sorgular ve en büyük, en küçük, ortalama, standart sapma gibi değişkenler. Sınırlı çalışma belleği akış hakkındaki özet bilgiyi tutmak için kullanılır. Özet bilgi ortalama, standart sapma, en büyük, en küçük gibi betimsel istatistik (descriptive statistics) bilgisidir. Bu bellek fiziksel bellek olabilir ya da hızlı bir NoSQL veritabanı olabilir.

Hoca bu kısmı şöyle toparladı:

- Akış veri yönetim sistemi veriyi operasyonel amaçla değil, arşiv amacıyla saklar.
- İki sorgu arayüzü vardır: anlık sorgular ve sürekli sorgular.
- Gelen akış hakkında özet bilgiyi tutmak için sınırlı bir çalışma belleği vardır.

7. Uygulamalar: Sorgu Akışları, Tıklama Akışları, IP Paketleri

7.1 Sorgu Akışlarında Madencilik

Google, hangi sorguların dünden daha sık arandığını bilmek isteyebilir. Google'a her gün milyonlarca insandan sürekli sorgu gelir. Bunların hepsi bir akış verisi oluşturur; akışın her elemanı bir sorgudur, bir anahtar kelimedir. "Hangi sorgular dünden daha sık?" sorusu bir sayma işlemine dönüşür: Karşılaşılan her sorgu için her gün bir sayaç tutulur ve günler karşılaştırılır.

7.2 Tıklama Akışlarında Madencilik

Yahoo, son bir saatte hangi sayfaların olağandışı sayıda tıklandığını (hit) bilmek isteyebilir. Milyonlarca kullanıcı siteyi sürekli kullanır ve farklı bağlantılara tıklar. Tıklama akışı (click stream) uygulamasında her bağlantının kaç kez tıklandığı sayılır ve her saat olağandışı sayıda tıklananlar bulunur.

7.3 IP Paketlerinin İzlenmesi

Anahtarlardan (switch) ve yönlendiricilerden (router) geçen IP paketlerinin sayısı çok büyüktür ve sürekli gelir, çünkü hepimiz interneti kullanıyoruz ve TCP/IP paketleri sürekli dolaşıyor. Bu paketler izlenerek sayma yapılabilir ya da en büyük, en küçük, standart sapma, ortalama gibi betimsel istatistikler hesaplanabilir. Bu özet bilgiler en iyi yönlendirme (optimal routing) için ve hizmet engelleme (denial of service) saldırılarını tespit etmek için kullanılır.

Bütün bu uygulamalarda amaç aynıdır: Her bir akış hakkında özet bilgi çıkarmak ve bu bilgiyi farklı uygulamalarda kullanmak. Bunun için farklı algoritmalar vardır.

8. Kayan Pencere

Akıştan bilgi çıkarmak için yararlı bir model pencere (window) kullanmaktır. Buna kayan pencere (sliding window) denir. Pencere iki şekilde tanımlanabilir:

- **Eleman sayısıyla:** Pencereye düşen eleman sayısı belirlenir. Örnek: Sisteme gelen her 1000 veride özet bilgi çıkarılır.
- **Zaman aralığıyla:** Örnek: Her pencere 30 saniyedir. Gelen bütün veri 30 saniyelik aralıklarla analiz edilir.

Hocanın örneğinde pencere boyutu 6 idi. Sistemde 6 eleman birikene kadar beklenir. 6 eleman olunca özet bilgiyi çıkaran fonksiyonlar uygulanır: en büyük değer, ortalama, standart sapma. Sonra yeni bir veri beklenir. Yeni veri gelince pencerenin en eski verisi atılır.

Hoca bunu harflerden oluşan bir akışla gösterdi. Bir taraf geçmiş, diğer taraf gelecekti; yeni veri gelecek tarafından gelir. Pencerede o anda 6 eleman vardı. Yeni eleman `k` gelince pencerenin en eski elemanı `s` pencereden düştü ve `k` pencereye eklendi. Ardından `l` gelince `d` düştü. Kısacası her yeni veri pencereye eklenir ve en eski eleman pencereden çıkar.

Bütün hesaplar fiziksel bellekte yapıldığı için pencere fiziksel belleğe sığacak boyutta olmalıdır. Veri kümelerini bellekte tutacaksak toplam boyutları fiziksel bellekten küçük olmalıdır. Boyutu 6 olan pencere yeterince küçüktür.

9. Pencere Belleğe Sığmazsa: Ortalamayı Güncellemek

Pencerenin uzunluğu N olsun. Akış sınırsızdır ve sürekli gelir. N o kadar büyük olsun ki pencere ana belleğe sığmasın. Ne yapılır?

Bu durumda pencerenin kendisi değil, pencereye ait özet bilgi fiziksel bellekte tutulur ve her yeni veri geldiğinde bu özet bilgi güncellenir.

Hocanın örneği penceredeki elemanların ortalamasıydı. Örnekteki akış harflerden oluştuğu için hoca bunları tam sayı gibi düşünmemizi istedi. Adımlar:

- İlk N eleman gelene kadar beklenir (örnekte 6).
- Pencere dolunca ortalama için toplam (sum) ve sayı (count) hesaplanır.
- Yeni bir eleman i geldiğinde pencerenin en eski elemanı j düşer. Ortalamaya şu değer eklenir:

$$\text{ort}_{\text{yeni}} = \text{ort} + \frac{i - j}{N}$$

Örnekte `k` eklenirken `s` düşüyordu; burada `k` i'ye, `s` j'ye karşılık gelir. Hoca bunu Türkçe de tekrar etti: Yeni bir veri geldiğinde bu verinin değerinden pencereden çıkan verinin değeri çıkarılır ve pencere boyutuna bölünür.

Aynı fikir standart sapma ve diğer betimsel istatistikler için de uygulanabilir. Bu istatistikler, pencerenin tamamı fiziksel belleğe sığmasa bile veriden yararlı bilgi çıkarmaya yardım eder.

10. Bloom Filtresi: Problem ve Fikir

Aradan sonra ilk algoritmaya geçildi: Bloom filtresi (Bloom filter). Bir akıştaki elemanlardan hangilerini daha önce gördüğünüzü anlamak istiyorsanız Bloom filtresine ihtiyaç duyabilirsiniz.

10.1 Web Tarayıcısı Örneği

Bir web tarayıcısını (web crawler) düşünelim. Tarayıcı web'de dolaşır. Bir URL ile karşılaştığında o URL'nin HTML sayfasını indirir, sayfanın içindeki URL'leri çıkarır ve bunları bir sonraki adımda ziyaret edilecekler olarak bir kuyruğa (frontier list) ekler. Sonra bu yeni URL'leri izleyerek yeni sayfalara erişir ve bunu sürdürür.

Sorun şudur: Bir URL'nin daha önce ziyaret edilip edilmediğini kontrol etmenin bir yolu yoksa tarayıcı aynı sayfayı tekrar tekrar indirebilir. Bu, Bloom filtresinin kullanılabileceği tipik bir problemdir. Akışta belirli bir elemanı daha önce görüp görmediğinizi bulmanız gerekiyorsa kullanmanız gereken algoritma Bloom filtresidir.

URL örneğinde tarayıcının gezdiği URL'ler sisteme bir akış olarak gelir. URL Bloom filtresine eşlenir. Filtre URL'nin daha önce görüldüğünü söylerse URL tekrar taranmaz; görülmediğini söylerse taranır.

10.2 Yapı

Bloom filtresi iki parçadan oluşur:

- Bir bit dizisi (bit array). Bu diziye Bloom filtresi denir ve fiziksel bellekte tutulur. Başta dizinin bütün bitleri 0'dır.
- Hash fonksiyonları. Her biri akıştan gelen elemanı argüman olarak alır ve bit dizisinde bir indeks üretir.

Akıştan her yeni eleman geldiğinde filtre bu elemanı hash fonksiyonlarıyla dizinin hücrelerine eşler ve o hücreleri 1 yapar.

11. Bloom Filtresine Eleman Ekleme: Oyuncak Örnek

Hoca bir oyuncak örnek (toy example) üzerinden gitti. Örnekte iki hash fonksiyonu vardı; hoca daha fazla hash fonksiyonu da olabileceğini söyledi. Bit dizisinin boyutu 11'dir. Bu yüzden hash fonksiyonları 0 ile 10 arasında bir indeks üretir.

- $h_1(x)$: x'i ikili (binary) gösterime çevir. Tek numaralı bitleri al. Bunları bir tam sayıya çevir. Sonucun mod11'ini al.
- $h_2(x)$: Aynı işlemi çift numaralı bitlerle yap.

Akışta sırasıyla 25, 159 ve 585 gelir.

11.1 Birinci Eleman: 25

- $25 = 11001_2$.
- Tek numaralı bitler 1., 3. ve 5. bitlerdir: `101`. $101_2 = 5$. $5 \bmod 11 = 5$. Dizinin 5 numaralı hücresi (0'dan saymaya başlayarak 0, 1, 2, 3, 4, 5) 1 yapılır.
- Çift numaralı bitler 2. ve 4. bitlerdir: `10`. $10_2 = 2$. $2 \bmod 11 = 2$. 2 numaralı hücre 1 yapılır.

İlk elemandan sonra dizi şöyledir:

```
indeks: 0 1 2 3 4 5 6 7 8 9 10
bit   : 0 0 1 0 0 1 0 0 0 0 0
```

11.2 İkinci Eleman: 159

- $159 = 10011111_2$.
- Tek numaralı bitler: `0111`. $0111_2 = 7$. $7 \bmod 11 = 7$. 7 numaralı hücre 1 yapılır.
- Çift numaralı bitler: `1011`. $1011_2 = 11$. $11 \bmod 11 = 0$. 0 numaralı hücre 1 yapılır.

```
indeks: 0 1 2 3 4 5 6 7 8 9 10
bit   : 1 0 1 0 0 1 0 1 0 0 0
```

11.3 Üçüncü Eleman: 585

Hoca üçüncü eleman için de aynı işlemin tekrarlandığını söyledi, ama hesabı altyazıda adım adım anlatmadı. Aşağıdaki adımlar aynı kurala göre hesaplandı (ara adım eklendi):

- $585 = 1001001001_2$.
- Tek numaralı bitler: `01001`. $01001_2 = 9$. $9 \bmod 11 = 9$.
- Çift numaralı bitler: `10010`. $10010_2 = 18$. $18 \bmod 11 = 7$. 7 numaralı hücre zaten 1'dir.

```
indeks: 0 1 2 3 4 5 6 7 8 9 10
bit   : 1 0 1 0 0 1 0 1 0 1 0
```

Üç eklemeden sonra 0, 2, 5, 7 ve 9 numaralı hücreler 1'dir; dizide 5 tane 1 vardır. Hoca da sorgu örneğinde dizideki 1'leri "1, 2, 3, 4, 5" diye saydı. Yeni tam sayılar geldikçe filtre aynı şekilde güncellenmeye devam eder.

> **Ders sonrası not:** Hoca "tek ve çift numaralı bitler" dedi ama numaralamanın hangi uçtan başladığını söylemedi. 159 için verdiği değerlerin (h_1 için `0111`, h_2 için `1011`) tutması için bitler sağdan, yani en düşük anlamlı bitten, 1'den başlayarak numaralanmalıdır. Soldan numaralansaydı tek bitler `1011` olurdu. 25 için iki numaralama aynı sonucu verir. Bütün değerler python ile doğrulandı.

12. Bloom Filtresinde Sorgulama ve Yanlış Pozitif

12.1 Sorgu Kuralı

Bir y elemanının filtrede olup olmadığını öğrenmek için y aynı hash fonksiyonlarına verilir. Her hash fonksiyonunun sonucu olan indekse gidilir ve oradaki bitin 1 mi 0 mı olduğuna bakılır.

- **Bitlerden en az biri 0 ise:** Filtre y'yi daha önce görmemiştir. Görmüş olsaydı o hücre 1 olurdu. Bu cevap %100 kesindir.
- **Bütün bitler 1 ise:** Filtre y'yi daha önce görmüş olabilir. Ama tam emin değiliz.

  Ders sonrası not: Hoca önce "değer iki hash fonksiyonu için de 0 ise görülmemiştir" dedi. Hemen ardından doğru kuralı verdi: Konumlardan en az biri 0 ise eleman görülmemiştir. Tek bir 0 yeterlidir.

12.2 Neden Kesin Değil?

Hash fonksiyonları kullandığımız için, sorguladığımız eleman başka elemanların daha önce 1 yaptığı indekslere eşlenebilir. Bu durumda 1 değerini görür ve "daha önce gördüm" sonucuna varırız; oysa aslında görmemişizdir. Bu bir yanlış pozitiftir (false positive).

> **Hocanın vurgusu:** Bloom filtresinde yanlış pozitif olabilir, ama yanlış negatif (false negative) hiçbir zaman olmaz. Filtre "görmedim" diyorsa %100 doğrudur. "Gördüm" diyorsa bazı durumlarda bu bir yanlış pozitiftir.

12.3 Sorgu Örneği: 118

Hoca aynı 11 bitlik filtre üzerinde bir sorgu yaptı: 118 elemanı akışta daha önce görüldü mü?

- 118 ikiliye çevrilir: $118 = 1110110_2$.
- Tek numaralı bitler `1110` $= 14$; $14 \bmod 11 = 3$. Hoca $h_1(118) = 3$ dedi.
- Çift numaralı bitler `101` $= 5$; $5 \bmod 11 = 5$. Hoca $h_2(118) = 5$ dedi.
- 3 numaralı hücre 0'dır, 5 numaralı hücre 1'dir.
- Bitlerden biri 0 olduğu için cevap negatiftir: 118 filtre tarafından daha önce görülmemiştir ve bu %100 kesindir.

İki değer de 1 olsaydı filtre "gördüm" derdi ama %100 emin olamazdı.

12.4 Basit Tahmin: 1'lerin Oranı

Yanlış pozitif olasılığı 1'lerin yoğunluğuna (density) bağlıdır. Yanlış pozitif, iki hash fonksiyonunun da 1 olan bir indeksi göstermesiyle olur. Filtrede 1'ler çoksa yanlış pozitif de çok olur.

Bir hash fonksiyonu için yanlış pozitife denk gelme olasılığı, filtredeki 1'lerin oranıdır. Örnekte 5 tane 1 ve 11 hücre vardır, yani bir hash için olasılık $5/11$'dir. İki hash fonksiyonu olduğu için iki olasılık çarpılır:

$$\left(\frac{5}{11}\right)^2 = \frac{25}{121} \approx 0.207$$

(Çarpımın sonucu derste söylenmedi; ara adım eklendi.)

12.5 İkiliye Çevirme Hakkında Soru

Soru: İkili sayıya çevirmeyi bilmemiz gerekiyor mu?

Cevap: Hoca bunun tartışmasız olduğunu, bir tam sayıyı ikiliye çevirmeyi bilmenin beklendiğini söyledi. Yöntemi de kısaca gösterdi: 2'nin kuvvetleri yazılır ($2^0, 2^1, \ldots, 2^6 = 64$). En büyük kuvvetten başlanır. Sayının içinde 64 varsa o basamağa 1 yazılır, kalan alınır ve bir sonraki kuvvetin kalanın içinde olup olmadığına bakılır. Böyle devam edilir.

13. Yanlış Pozitif Olasılığı: Dart Modeli

Hoca 1'lerin oranını formüle dökmek için bir dart benzetmesi kullandı.

13.1 Hedefler ve Dartlar

- Filtredeki 0'ları hedef (target) olarak düşünün. t hedef vardır; t bit dizisinin boyutudur. Başta hepsi 0'dır.
- Bu hedeflere dart atıyoruz. Dart sayısı d, akıştaki eleman sayısı ile hash fonksiyonu sayısının çarpımıdır. Çünkü oyuncak örnekte bir eleman iki hash fonksiyonuna verilir ve iki indeks günceller. Bütün akış için filtrenin güncellenme sayısı, eleman sayısı çarpı hash sayısıdır.

13.2 Türetme

- Bir dartın belirli bir hedefi vurma olasılığı $1/t$'dir; önünüzde t farklı seçenek vardır.
- Bir dartın o hedefi vurmama olasılığı $1 - 1/t$'dir.
- d dart atıldığında, yani d deneme yapıldığında, olasılıklar çarpılır. Hiçbir dartın belirli bir hedefi vurmama olasılığı:

$$\left(1 - \frac{1}{t}\right)^d$$

- Hoca bu ifadeyi üssü $t \cdot \dfrac{d}{t}$ biçiminde yazarak yeniden düzenledi:

$$\left(1 - \frac{1}{t}\right)^d = \left(\left(1 - \frac{1}{t}\right)^t\right)^{d/t}$$

- İçteki ifade yaklaşık $1/e$'dir. Bu yüzden bütün ifade şuna dönüşür:

$$\left(1 - \frac{1}{t}\right)^d \approx e^{-d/t}$$

Ders sonrası not: Altyazıda bu sabit "epsilon" diye geçiyor. Doğrusu Euler sayısı $e \approx 2.718$'dir; $\left(1 - \frac{1}{t}\right)^t$ ifadesi büyük t için $1/e$'ye yaklaşır.

Ek bilgi (derste anlatılmadı): $\lim_{t\to\infty} \left(1 - \frac{1}{t}\right)^t = \frac{1}{e}$ eşitliği, e sayısının bilinen limit tanımından gelir. Bloom filtrelerinde t milyonlar ya da milyarlar mertebesinde olduğu için yaklaşıklık çok iyidir.

13.3 0'ların Oranından Yanlış Pozitife

$e^{-d/t}$, hiçbir dartın belirli bir hedefi vurmama olasılığıdır. Bu da filtredeki 0'ların oranıdır. Bunu 1'den çıkarınca 1'lerin yoğunluğu bulunur: $1 - e^{-d/t}$. Daha önce söylendiği gibi 1'lerin yoğunluğu yanlış pozitifi verir. Hash fonksiyonu sayısı k ise:

$$P(\text{yanlış pozitif}) \approx \left(1 - e^{-d/t}\right)^k$$

Ek bilgi (derste anlatılmadı): Kaynaklarda bu formül genellikle m eleman, n bit ve k hash fonksiyonu ile $\left(1 - e^{-km/n}\right)^k$ biçiminde yazılır. Dersteki gösterimle $t = n$ ve $d = k \cdot m$ olduğu için iki yazım aynıdır.

13.4 Sayısal Örnek

Hoca somut bir örnek verdi:

- **Verilen:** Bloom filtresi 1 milyar bitten oluşur: $t = 10^9$. 5 hash fonksiyonu vardır. 10^8 (100 milyon) eleman eklenecektir.
- **Dart sayısı:** $d = 5 \times 10^8$.
- **Oran:** $d/t = \dfrac{5 \times 10^8}{10^9} = 0.5$ (ara adım eklendi).
- **0'ların oranı:** $e^{-0.5} \approx 0.607$. Hoca bunun hiçbir dartın belirli bir hedefi vurmama olasılığı, yani filtredeki 0'ların oranı olduğunu söyledi.
- **1'lerin oranı:** $1 - 0.607 = 0.393$.
- **Yanlış pozitif olasılığı:** 1'lerin oranının hash sayısı kadar kuvveti: $0.393^5 \approx 0.0094$.

  Ders sonrası not: Hocanın 0.393^5 sonucunu söylediği kısım altyazıda bozuk. Python ile hesaplanan değer $0.3935^5 \approx 0.00943$, yani yaklaşık %0.94'tür. $\left(1 - e^{-km/n}\right)^k$ formülüne $k = 5$, $m = 10^8$, $n = 10^9$ koyunca da aynı sonuç çıkar.

Hoca bu bölümü şöyle bağladı: Bloom filtresine değer eklemeyi, filtreyi güncellemeyi, filtrede sorgu yapmayı ve filtre boyutu, hash fonksiyonu sayısı ve eklenecek eleman sayısı verildiğinde yanlış pozitif olasılığını hesaplamayı gördük.

14. Son k Bitteki 1'leri Saymak

Bloom filtresi, bir elemanın daha önce görülüp görülmediğini bulma problemi içindi. Şimdi problem farklıdır: Bir ikili akışta (binary stream) gelen 1'leri saymak (counting ones).

Sürekli gelen bir ikili akış düşünün. Bir taraf geçmiş, diğer taraf gelecektir; her yeni bit gelecek tarafından gelir. Sorulan soru şudur: Son k bitte kaç tane 1 var? N pencere boyutudur ve $k \leq N$'dir. Soru sorulduğunda bitleri tek tek saymak istemeyiz. Bunun yerine şu ana kadar görülen 1'leri saymak için kullanılabilecek bir özet bilgi oluşturmak isteriz.

14.1 Uygulama: Spam Filtresi

Hocanın örneği spam filtresiydi. Sisteme gelen bütün e-postalar spam ya da spam değil diye etiketlenir. Normal e-posta 0, spam e-posta 1 ile gösterilirse ikili bir akış oluşur. "Son 100.000 e-postada kaç spam gördüm?" sorusunu cevaplamak için akıştaki 1'leri sayan bir algoritma gerekir. Hoca bunun yalnız bir örnek olduğunu, başka uygulamaların da bulunduğunu söyledi.

14.2 Açık Çözüm Neden Uygun Değil?

Açık çözüm, N biti fiziksel bellekte saklamak ve soru gelince son k biti saymaktır. Bu iki sebeple uygun değildir:

- Pencere boyutu genellikle büyüktür ve veri fiziksel belleğe sığmaz; bellek gereksinimi çok büyük olur.
- Belleğe sığsa bile her soruda son k bitteki 1'leri saymak istemediğimiz bir zaman karmaşıklığı getirir. Cevabı hemen vermek isteriz.

Hoca önce temel fikri, sonra bu fikrin iyileştirilmiş hâli olan DGIM algoritmasını anlatacağını söyledi.

15. İlk Fikir: Üstel Büyüyen Bloklar

Fikir, akışı bloklarla özetlemektir. Akışın üstel olarak büyüyen bloklarını özetleyen ek bir veri yapısı kurulur. Kurallar:

- Herhangi bir boyuttan ikiden fazla blok olmaz.
- Küçük bloklar, daha büyük bir bölgeyle aynı noktada başlıyorsa atılır.

Hocanın gösterdiği şekilde akış 0 ile başlıyor, sonra 1, sonra 0 diye devam ediyordu. Akış geldikçe algoritma blokları kuruyordu. Her blok belirli sayıda biti tanımlar ve yukarı çıkıldıkça bloklar üstel olarak büyür: 1 bitlik bloklar, 2 bitlik bloklar, 4 bitlik bloklar ve böyle devam eden $2^0, 2^1, 2^2, 2^3, 2^4$ boyutları. Örneğin en üstteki bir blok 16 biti, bir altındaki 8 biti, bir altındaki 4 biti kapsar. Her bloğun içine o bloktaki 1 sayısı yazılır. Hoca şekilde 1 bitlik bloklar için 0 ve 1, 2 bitlik bloklar için 1 ve 2, 4 bitlik bloklar için 3 ve 2 değerlerini okudu.

15.1 Son 28 Bitteki 1'ler

Soru: Son 28 bitte kaç tane 1 var?

Algoritma ilk bloktan başlar. Blokların kapsadığı bit sayılarını ve içlerindeki 1 sayılarını ayrı ayrı toplar. Hocanın anlatımına göre:

- Kapsanan bitler: $1 + 2 = 3$, $3 + 4 = 7$, $7 + 16 = 23$. Bu bloklar son 23 biti kapsar.
- Bu bloklardaki 1'ler: $10 + 2 = 12$, $12 + 1 = 13$, $13 + 0 = 13$. Son 23 bitte 13 tane 1 vardır.

Bunu yalnız veri yapısına bakarak hesaplayabiliriz. Ama soru 28 bitti; 5 bit daha gerekir ($28 - 23 = 5$). Sıradaki blok 16 biti kapsar ve içinde 6 tane 1 vardır. Bu bitleri tutmadık, yalnız özet bilgiyi tuttuk. Bu yüzden bir tahmin yapılır: 6 tane 1'in blokta düzgün dağıldığı varsayılırsa son 5 bite düşen 1 sayısı

$$\frac{5}{16} \times 6 = 1.875$$

olur (sonuç ara adım olarak eklendi). Bu durumda tahmin yaklaşık $13 + 1.875 \approx 14.9$ olur (ara adım eklendi).

15.2 Fikrin Kusuru

1'lerin çoğu ya da hepsi 16 bitlik bloğun diğer kısmında, yani istenmeyen 11 bitte olabilir. O zaman son 5 bitin hepsi 0'dır ve "bu bölgede 1'ler var" demek yanlış olur. Fikir yararlıdır ama kusurludur. DGIM algoritması bu sorunu çözer.

16. DGIM Algoritması

DGIM algoritmasının adı onu bulan yazarlardan gelir. Bu algoritma da bir veri yapısı kurar. Farkı şudur: Önceki fikirde bloklar bit sayısına göre kuruluyordu. DGIM'de bloklar (kovalar, buckets) yalnız 1'lere göre kurulur.

Ek bilgi (derste anlatılmadı): DGIM, algoritmayı yayımlayan Datar, Gionis, Indyk ve Motwani'nin soyadlarının baş harfleridir; yayın büyük olasılıkla 2002 yılındadır. Algoritma, N bitlik bir pencere için büyük olasılıkla $O(\log^2 N)$ bit bellek kullanır.

16.1 Tutulan Bilgi ve Kurallar

Algoritma iki şey tutar: kovalardan oluşan veri yapısı ve her biti gösteren zaman damgası (timestamp). Kurallar:

- Kova yalnız 1 ile karşılaşıldığında oluşturulur. 0 için hiçbir şey oluşturulmaz.
- Bir kovanın boyutu içindeki 1 sayısıdır ve 2'nin kuvvetidir. Boyutu 2^1 olan kovada 2 tane 1, boyutu 2^2 olan kovada 4 tane 1 vardır.
- Kova 1 ile başlar ve 1 ile biter.
- Aynı boyuttan en fazla iki kova olabilir.
- Kovalar örtüşmez.
- Kovalar boyuta göre sıralıdır.
- Bitiş zamanı N zaman biriminden daha eskiye düşen kova silinir.

Hoca kova boyutunun bit sayısı olmadığını bir örnekle gösterdi: Boyutu 2 olan bir kova `101` gibi 3 bit, boyutu 2 olan başka bir kova `1001` gibi 4 bit olabilir. İkisinde de 2 tane 1 vardır.

16.2 Yeni Bit Gelince

- En eski kovanın bitiş zamanı N zaman biriminden eskiye düştüyse o kova atılır.
- Yeni bit 0 ise başka değişiklik gerekmez.
- Yeni bit 1 ise boyutu 1 olan yeni bir kova oluşturulur.
- Boyutu 1 olan üç kova olduysa en eski ikisi boyutu 2 olan tek bir kovada birleştirilir.
- Bu birleştirme boyutu 2 olan üç kova oluşturduysa en eski ikisi boyutu 4 olan bir kovada birleştirilir. Bu, 8, 16 … boyutları için zincirleme devam eder.

16.3 Birleştirme Örneği

Hoca bir başlangıç durumu gösterdi ve akışa 1 geldiğini söyledi. Yeni 1 için boyutu 1 olan bir kova oluşturuldu. Daha önce boyutu 1 olan iki kova vardı; yenisiyle üç oldu. Algoritma ikiden fazla olunca öncekilerden ikisini daha büyük bir kovada birleştirmeyi söyler. En eski iki kova boyutu 2 olan bir kovada birleştirildi, yeni kova değişmedi.

Hocanın söylediğine göre bu adımdan sonra durum şöyleydi (yeniden eskiye):

```
kova boyutları: 1 | 2 2 | 4 4 | 8 8 | 16
```

Ardından akışa önce 0 geldi; 0 için hiçbir şey yapılmadı. Sonra yeni bir 1 geldi ve hocanın anlatımına göre birleştirmeler zincirleme ilerledi:

- Boyutu 1 olan üç kova oldu; en eski ikisi boyutu 2 olan kovada birleştirildi.
- Boyutu 2 olan üç kova oldu; en eski ikisi boyutu 4 olan kovada birleştirildi.
- Boyutu 4 olan üç kova oldu; en eski ikisi boyutu 8 olan kovada birleştirildi.
- Boyutu 8 olan üç kova oldu; en eski ikisi bir üst boyutta birleştirildi.

Son durumda boyutu 1, 2, 4 ve 8 olan kovalardan birer tane kaldı ve yapı böyle devam etti. Yeni bitler geldikçe DGIM veri yapısını sürekli günceller.

Ders sonrası not: Hocanın söylediği ara durumdan (boyutu 1 olan tek kova) sonra yalnız 0 ve 1 gelirse boyutu 1 olan iki kova olur ve hiçbir birleştirme gerekmez. Anlatılan zincirleme birleştirmenin olması için araya bir 1 daha girmelidir; kitabın slaytında büyük olasılıkla sıra "1, ardından 0, ardından 1" idir. Python simülasyonunda `1 | 2 2 | 4 4 | 8 8 | 16` durumuna 1, 0, 1 eklenince `1 | 2 | 4 | 8 | 16 16` sonucu çıkıyor; bu, hocanın söylediği son duruma (1, 2, 4, 8'den birer kova) uyar.

17. DGIM'de Sorgu ve Hata Sınırı

17.1 Sorgu

Son k bitteki 1 sayısı sorulduğunda (örnekte $k = 28$) algoritma veri yapısını ve zaman damgalarını kullanır. Zaman damgaları sayesinde 28. bitin nerede olduğunu kesin olarak bilir.

- Yalnız bitiş zaman damgası en fazla k zaman birimi önce olan kovalar dikkate alınır.
- En eski kova hariç bu kovaların boyutları toplanır.
- k sınırının içine düştüğü en eski kova için boyutunun yarısı alınır ve toplama eklenir.

Hoca bunu Türkçe de söyledi: Sınıra kadar olan bütün blokların içindeki sayısal değerler toplanır. Sorulan k değerinin içine düştüğü bir blok varsa o bloğun yarısı alınır.

17.2 Hata Sınırı

Bu yöntem en fazla %50 hata verir. Hoca gerekçeyi şöyle anlattı: Kovaların boyutları $2^0, 2^1, 2^2, 2^3, 2^4$ olsun ve en eski kova 2^5 boyutunda olsun. Daha küçük kovaların toplamı

$$2^0 + 2^1 + 2^2 + 2^3 + 2^4 = 31 = 2^5 - 1$$

olur. Yani küçük kovaların toplamı neredeyse en eski kova kadardır. Sorgudaki k değeri en eski kovanın içine düşerse algoritma bu kovanın yarısını, yani $2^5/2 = 16$'yı alır. Yapılabilecek en büyük hata bu kovanın yarısı kadardır; bu da %50'dir.

> **Hocanın vurgusu:** DGIM'de büyük kova boyutları için en büyük hata %50'dir. Bu oran çok küçük değerlere indirilebilir, ama bunun için veri yapısı ve zaman damgalarına ek olarak fiziksel bellekte daha fazla bilgi tutmak gerekir.
>
> **Ek bilgi (derste anlatılmadı):** Kitapta bu iyileştirme, her boyuttan en fazla iki kova yerine r ya da $r - 1$ tane kova tutulmasına izin verilerek yapılır; hata oranı büyük olasılıkla $1/(r - 1)$ mertebesine iner. Bedeli, daha fazla kova tutmaktır.

Hoca örnekleme (sampling) konusunun gelecek haftaya kaldığını söyledi.

18. Dönem Projesinin Teknik Konusu

Dersin son kısmında dönem projesi anlatıldı. Projenin teknik içeriği şöyledir:

- **Çatı:** Apache Hadoop ya da Spark kullanılabilir. İkisi de MapReduce programlamaya, yani Map ve Reduce işleri (jobs) yazmaya izin veren kütüphanelerdir. Hangisinin seçildiği önemli değildir.
- **HDFS zorunluluğu:** Uygulama mutlaka HDFS dosya sistemi üzerinde çalışmalıdır.
- **Küme kurulumu:** Kendi bilgisayarınızda VirtualBox ile sanal makineler oluşturup HDFS düğümlerini bunlara kurarsınız. Üç düğüm olur: fiziksel bilgisayar ve iki sanal makine. Biri ad düğümü (name node), diğer ikisi işçi düğüm (worker node) olur. Bu üç düğüm birbiriyle konuşan bir HDFS kümesi (cluster) kurar. Spark ya da Hadoop kodu bu kümenin üzerinde çalıştırılır.
- **Veri kümesinin boyutu:** Belleğe sığmayacak büyüklükte bir veri kümesi beklenmez. Amaç MapReduce tabanlı program yazabildiğinizi ve bunu bir büyük veri işleme ortamında çalıştırabildiğinizi göstermektir. Hocanın örneği: 16 GB belleği olan bir dizüstü bilgisayarda Kaggle ya da başka bir veri sağlayıcısından alınmış 1 GB'lık bir veri kümesi.

Projenin iki kısmı vardır:

- **HDFS kısmı:** Bir grafik kullanıcı arayüzü (GUI) yazılır. Bu arayüz yerel dosya sisteminden bir dosya seçip HDFS'ye yüklemeyi sağlar. Bunun için dosyayı HDFS'ye ekleyen bir sınıf yazılır. Arayüz ayrıca HDFS'de o an bulunan dosyaları da listelemelidir.
- **MapReduce kısmı:** HDFS'ye yüklenen veri kümesi üzerinde MapReduce programlama modeliyle beş farklı istatistik fonksiyonu yazılır ve çalıştırılır. Hocanın örnekleri: aralık (range), standart sapma, en küçük, en büyük ve ortalama.

Ardından yazılan fonksiyonların performans analizi yapılır. Hocanın özetiyle proje, HDFS kurmayı, Spark ya da Hadoop gibi büyük veri işleme sistemlerini öğrenmeyi, MapReduce programı yazmayı ve dağıtık bir sistemle etkileşen bir grafik arayüz yazmayı gerektiren bir MapReduce ve HDFS projesidir.

Örnek Sorular

Bu derste ekran kaydı ve kare yoktur. Sorular, hocanın derste sözlü sorduğu ve çözdüğü sorulardan alındı.

Soru 1 (derste sözlü, 00:36:40)

Bir akışta sırasıyla 1, 5, 2, 7 geliyor. "Şu ana kadar görülen en büyük değer nedir?" anlık sorgusu ve "Her yeni en büyük değeri bildir." sürekli sorgusu her adımda ne yapar?

Çözüm (derste verildi): Anlık sorguda bellekteki en büyük değer sırasıyla 1, 5, 5, 7 olur. Çıktı yalnız soru sorulduğunda verilir. Sürekli sorguda 5 ve 7 geldiğinde çıktı üretilir, 2 geldiğinde çıktı üretilmez. İlk eleman 1 de o ana kadarki en büyük değer olduğu için büyük olasılıkla çıktı üretir.

Soru 2 (derste sözlü, 01:21:26)

11 bitlik bir Bloom filtresi ve iki hash fonksiyonu var: h_1 ikili gösterimin tek numaralı bitlerini, h_2 çift numaralı bitlerini alıp tam sayıya çevirir ve mod11 alır. Akışta 25, 159 ve 585 geliyor. Filtrenin son hâli nedir?

Çözüm (25 ve 159 derste verildi): $h_1(25) = 5$, $h_2(25) = 2$. $h_1(159) = 7$, $h_2(159) = 0$.

> **Çözüm (derste verilmedi):** $585 = 1001001001_2$. Tek bitler `01001` $= 9$, $h_1 = 9$. Çift bitler `10010` $= 18$, $h_2 = 18 \bmod 11 = 7$. Filtrenin son hâli: 0, 2, 5, 7 ve 9 numaralı bitler 1, diğerleri 0. Sonuç python ile doğrulandı.

Soru 3 (derste sözlü, 01:31:22)

Aynı filtrede 118 daha önce görüldü mü?

Çözüm (derste verildi): $h_1(118) = 3$, $h_2(118) = 5$. 3 numaralı bit 0 olduğu için 118 kesin olarak görülmemiştir.

Soru 4 (derste sözlü, 01:35:21)

Aynı filtrede 11 bitin 5'i 1 iken iki hash fonksiyonu için yanlış pozitif olasılığı yaklaşık nedir?

Çözüm (derste verildi): Bir hash için olasılık $5/11$'dir, iki hash için iki olasılık çarpılır: $(5/11)^2 \approx 0.207$ (son sayı ara adım olarak eklendi).

Soru 5 (derste sözlü, 01:43:33)

Bloom filtresi 10^9 bittir. 5 hash fonksiyonu vardır ve 10^8 eleman eklenecektir. Yanlış pozitif olasılığı nedir?

Çözüm (derste verildi): $d = 5 \times 10^8$, $d/t = 0.5$. 0'ların oranı $e^{-0.5} \approx 0.607$, 1'lerin oranı 0.393. Yanlış pozitif olasılığı $0.393^5 \approx 0.0094$.

Soru 6 (derste sözlü, 02:00:34)

Üstel bloklarla özetlenmiş bir akışta son 28 bitteki 1 sayısını tahmin edin. Son 23 biti kapsayan bloklarda 13 tane 1 var. Sıradaki blok 16 biti kapsıyor ve içinde 6 tane 1 var.

Çözüm (derste verildi): Eksik bit sayısı $28 - 23 = 5$. 1'lerin düzgün dağıldığı varsayılırsa sıradaki bloktan $\frac{5}{16} \times 6 = 1.875$ tane 1 gelir. Toplam tahmin yaklaşık $13 + 1.875 \approx 14.9$'dur (son iki sayı ara adım olarak eklendi). Hoca bu tahminin, 1'ler bloğun diğer ucunda toplanmışsa çok yanlış olabileceğini söyledi.

Dersin Toparlaması

Bu derste 4. bölüme, veri akışlarında madenciliğe başlandı. Akış verisinin ilişkisel veritabanından farkı, giriş ve giriş hızı üzerinde kontrolümüzün olmamasıdır. Akış verisi dağıtık sistemlerde arşivlenir ama sorgulanmaz; asıl iş sınırlı bellekte tutulan özet bilgilerle yapılır. Bu özet bilgiye iki arayüzden erişilir: bir kez sorulan anlık sorgular (istek-yanıt) ve sürekli çalışan sürekli sorgular (yayınla-abone ol). Kayan pencere, akışı sınırlı bir bölümde analiz etmeyi sağlar. Pencere belleğe sığmadığında ortalama gibi istatistikler $(i-j)/N$ güncellemesiyle tutulur.

İki algoritma görüldü. Bloom filtresi, bir elemanın daha önce görülüp görülmediğini bir bit dizisi ve hash fonksiyonlarıyla cevaplar; yanlış pozitif verebilir ama yanlış negatif vermez ve yanlış pozitif olasılığı dart modeliyle $\left(1-e^{-d/t}\right)^k$ olarak hesaplanır. DGIM algoritması, son k bitteki 1'leri boyutu 2'nin kuvveti olan ve her boyuttan en fazla iki tane bulunan kovalarla tahmin eder; en eski kovanın yarısı alındığı için hata en fazla %50'dir. Dersin sonunda HDFS ve MapReduce üzerine kurulu dönem projesinin teknik konusu anlatıldı. Hoca örnekleme konusunun ve bir bölümün daha gelecek hafta işleneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.